

EVIDENCIAS DE ENTREGA

API de Categorías con FastAPI

Jorge Alejandro Torres Paez

Servicio Nacional de Aprendizaje (SENA)

Tecnólogo en Análisis y Desarrollo de Software (ADSO) — Ficha N.º 3407180

Tercer Trimestre — Pruebas de Software, Módulo III

Instructor(a): Jeison Aly Contreras

Fecha de Entrega: 13/09/2026

Propósito del Documento

Este documento lo elaboré para organizar las evidencias que se piden en la actividad autónoma de la API de Categorías con FastAPI, del Módulo III de Pruebas de Software. En cada punto explico brevemente qué debe mostrar la imagen y dejo la captura de pantalla correspondiente, con el fin de dejar constancia del trabajo realizado durante la actividad.

Evidencias Fotográficas y Capturas de Pantalla

Captura de Swagger (/docs)

En esta parte pego la captura de la interfaz Swagger, que es la documentación que genera FastAPI de forma automática en la ruta /docs. Ahí se pueden ver todos los endpoints que desarrollé para la API de categorías.

Elementos que debe evidenciar la imagen:

- Se ve la URL /docs abierta en el navegador.
- Aparecen los seis endpoints que pide la actividad: GET /categories, GET /categories/{category_id}, POST /categories, PATCH /categories/{category_id}, DELETE /categories/{category_id} y el filtro ?active=.
- Se alcanzan a leer los métodos HTTP de cada endpoint (GET, POST, PATCH, DELETE).

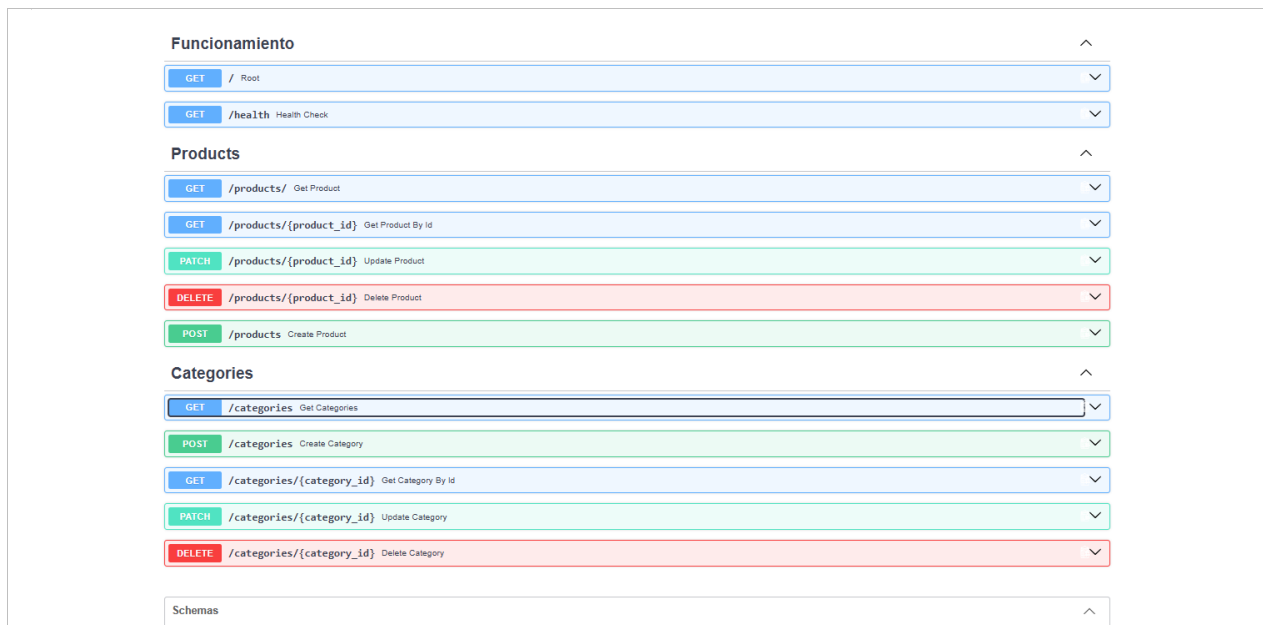


Figura 1 — Endpoints documentados en Swagger (/docs)

Captura de Ejecución de pytest -v

En esta evidencia pego la captura de la terminal donde se ve la ejecución completa de las pruebas automatizadas que hice con pytest, usando el comando `pytest -v`.

Elementos que debe evidenciar la imagen:

- Se ve el comando que ejecuté (`pytest -v`).
- Se alcanzan a leer los nombres de las pruebas que corrí (mínimo diez; en mi caso hice las doce de la matriz CA01 a CA12).
- Se ve el resultado final de la ejecución (PASSED o FAILED) y el resumen con la cantidad de pruebas.

```

PS C:\Users\Usuario\Desktop\Proyecto_MTP\test_fastapi_productos_y_categorias> pytest -v
===== test session starts =====
platform win32 -- Python 3.12.7, pytest 8.1.1, pluggy 1.6.0 -- C:\Users\Usuario\Desktop\Proyecto_MTP\test_fastapi_productos_y_categorias\.venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\Usuario\Desktop\Proyecto_MTP\test_fastapi_productos_y_categorias
configfile: pytest.ini
collected 23 items

test/test_categories.py::test_list_categories PASSED [ 40%]
test/test_categories.py::test_get_existing_category PASSED [ 60%]
test/test_categories.py::test_get_non_existing_category PASSED [ 80%]
test/test_categories.py::test_get_invalid_id PASSED [ 85%]
test/test_categories.py::test_create_valid_category PASSED [ 91%]
test/test_categories.py::test_create_category_name_too_short PASSED [ 95%]
test/test_categories.py::test_create_category_missing_name PASSED [ 97%]
test/test_categories.py::test_update_existing_category PASSED [ 98%]
test/test_categories.py::test_update_non_existing_category PASSED [ 99%]
test/test_categories.py::test_delete_existing_category PASSED [ 100%]
test/test_categories.py::test_delete_non_existing_category PASSED [ 100%]
test/test_categories.py::test_filter_active_categories PASSED [ 100%]
test/test_categories.py::test_search_categories PASSED [ 100%]
test/test_categories.py::test_search_categories_no_match PASSED [ 100%]
test/test_products.py::test_health PASSED [ 100%]
test/test_products.py::test_get_products PASSED [ 100%]
test/test_products.py::test_get_existing_product PASSED [ 100%]
test/test_products.py::test_get_non_existing_product PASSED [ 100%]
test/test_products.py::test_create_product PASSED [ 100%]
test/test_products.py::test_update_product PASSED [ 100%]
test/test_products.py::test_update_product_stock_recalculates_available PASSED [ 100%]
test/test_products.py::test_delete_product PASSED [ 100%]
test/test_products.py::test_filters PASSED [ 100%]

===== warnings summary =====
venv\Lib\site-packages\starlette\testclient.py:33
  C:\Users\Usuario\Desktop\Proyecto_MTP\test_fastapi_productos_y_categorias\.venv\Lib\site-packages\starlette\testclient.py:33: DeprecationWarning: The anyio.bac.BlockingPortal alias is deprecated, use anyio.from_thread.BlockingPortal instead.
    _portal_factory_type = Callable[[], AbstractContextManager[AnyioABC.BlockingPortal]]

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 23 passed, 1 warning in 0.19s =====

```

Figura 2 — Ejecución completa de la suite con `pytest -v`

Reto Opcional

Si alcancé a desarrollar la búsqueda por nombre (`GET /categories?search=comp`), en esta parte dejo la evidencia de que funciona correctamente, junto con las pruebas automatizadas que hice para esa funcionalidad.

Captura del Reto Opcional — Filtro de Búsqueda

Pego aquí la captura de Swagger donde se ve que el parámetro de búsqueda por nombre funciona, y las pruebas automatizadas que hice para esta parte.

Elementos que debe evidenciar la imagen:

- Se ve la petición `GET /categories?search=...` y la respuesta que devuelve.
- Se ven mínimo dos pruebas automatizadas relacionadas con esta funcionalidad.

The image shows the Swagger UI interface for an API. The top section is titled "Categories" and shows a GET request to the endpoint `/categories` with the description "Get Categories". The "Parameters" section is expanded, showing two query parameters: `active` (a dropdown menu) and `search` (a text input field containing "Mice"). Below the parameters are "Execute" and "Clear" buttons. The "Responses" section is also expanded, showing a 200 status code. The "Response body" is displayed as a JSON object:

```
{  "name": "Mice",  "description": "Ratón para computadora.",  "active": false,  "id": 7}
```

. The "Response headers" section shows:

```
content-length: 80content-type: application/jsondate: Sun, 13 Sep 2020 22:05:40 GMTserver: uvicorn
```

. At the bottom, there is a "Responses" table with columns "Code" and "Description", and a "Links" section.

Categories

GET /categories Get Categories

Parameters

Cancel

Name Description

active
boolean |
(boolean |
null)
(query)

search
string |
(string |
null)
(query)

Mice

Execute Clear

Responses

Curl

```
curl -X 'GET' \  
  'http://127.0.0.1:8000/categories?search=Mice' \  
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/categories?search=Mice
```

Server response

Code Details

200

Response body

```
{  "name": "Mice",  "description": "Ratón para computadora.",  "active": false,  "id": 7}
```

Response headers

```
content-length: 80content-type: application/jsondate: Sun, 13 Sep 2020 22:05:40 GMTserver: uvicorn
```

Responses

Code Description

Links

Figura 3 — Búsqueda por nombre (?search=Mice) en Swagger

Casos de Prueba (CA01–CA12)

CA01 — Listar categorías

Verifica que el endpoint devuelva la lista completa de categorías en formato JSON.

Elementos que debe evidenciar la imagen:

- Se ve la petición GET /categories.
- Se ve el código 200 y el arreglo JSON con las categorías registradas.

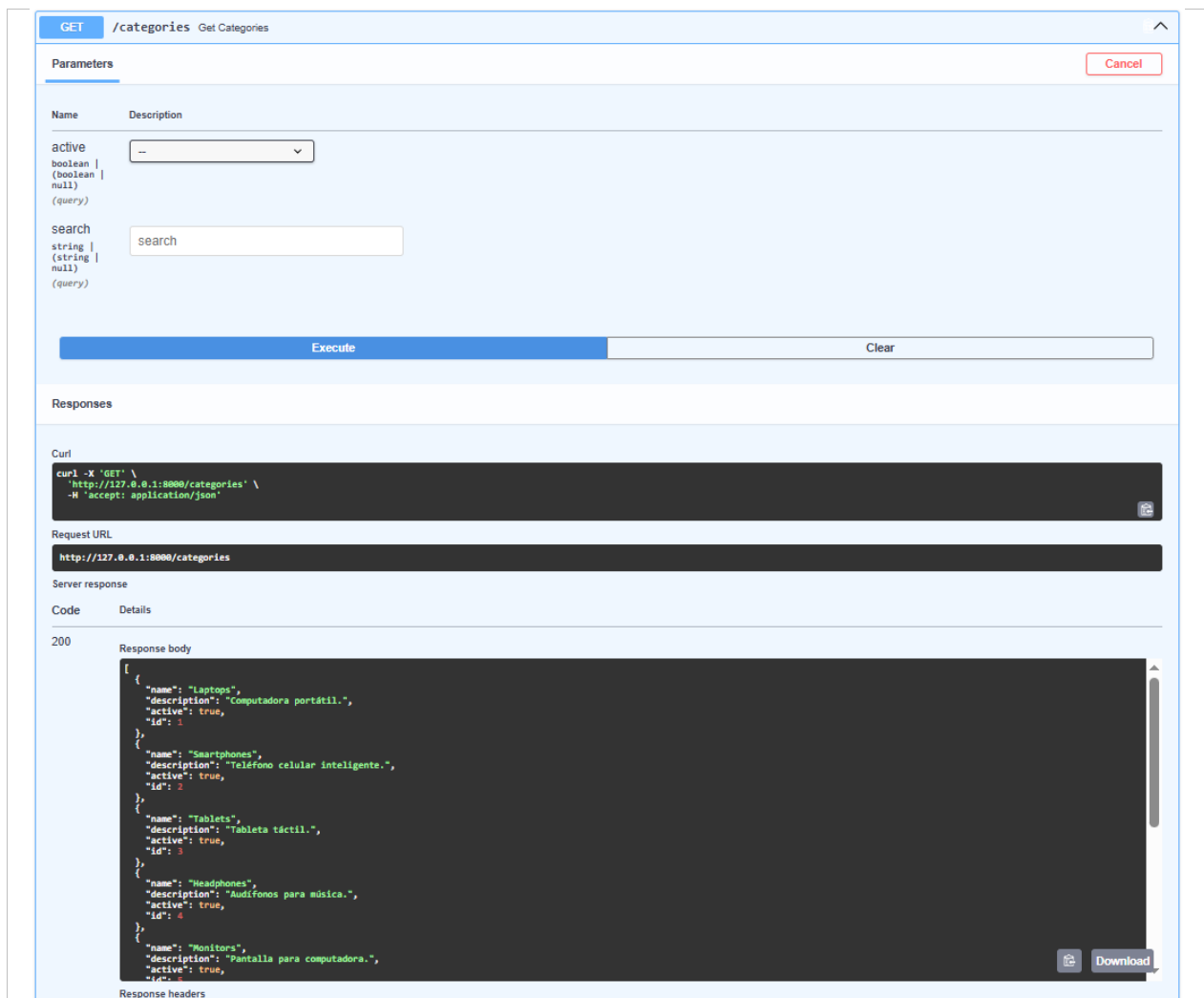


Figura — CA01

CA02 — Consultar existente

Verifica la consulta de una categoría por un id que sí existe en la base de datos.

Elementos que debe evidenciar la imagen:

- Se ve la petición GET /categories/1.

- Se ve el código 200 con los datos de la categoría solicitada.

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** /categories/{category_id} (Get Category By Id)
- Parameters:**
 - category_id:** Required, integer (path), value: 1
- Buttons:** Execute, Clear, Cancel
- Responses:**
 - Curl:** `curl -X 'GET' \ 'http://127.0.0.1:8000/categories/1' \ -H 'accept: application/json'`
 - Request URL:** `http://127.0.0.1:8000/categories/1`
 - Server response:**
 - Code:** 200
 - Response body:** `{ "name": "Laptops", "description": "Computadora portátil.", "active": true, "id": 1 }`
 - Response headers:** `content-length: 78, content-type: application/json, date: Sun, 13 Sep 2026 22:13:39 GMT, server: uvicorn`
- Summary Table:**

Code	Description	Links
200	Successful Response	No links
422	Validation Error	No links

Figura — CA02

CA03 — Consultar inexistente

Verifica el manejo de error al consultar un id de categoría que no existe (id = 0).

Elementos que debe evidenciar la imagen:

- Se ve la petición GET /categories/0.
- Se ve el código 404 con el detalle "Category not found".

GET /categories/{category_id} Get Category By Id

Parameters

Name	Description
category_id * required integer (path)	0

Execute **Clear**

Responses

404 Error: Not Found
Undocumented

Response body

```
{
  "detail": "Category not found"
}
```

Response headers

```
content-length: 31
content-type: application/json
date: Sun, 13 Sep 2026 22:16:24 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type
application/json

Example Value | Schema

```
{
  "name": "Laptops",
  "description": "computadora portátil.",
  "active": true,
  "id": 0
}
```

422 Validation Error
No links

Figura — CA03

CA04 — ID inválido

Verifica la validación de Pydantic cuando el id no es un número entero.

Elementos que debe evidenciar la imagen:

- Se ve la petición GET /categories/abc.
- Se ve el código 422 indicando que el valor debe ser un entero.

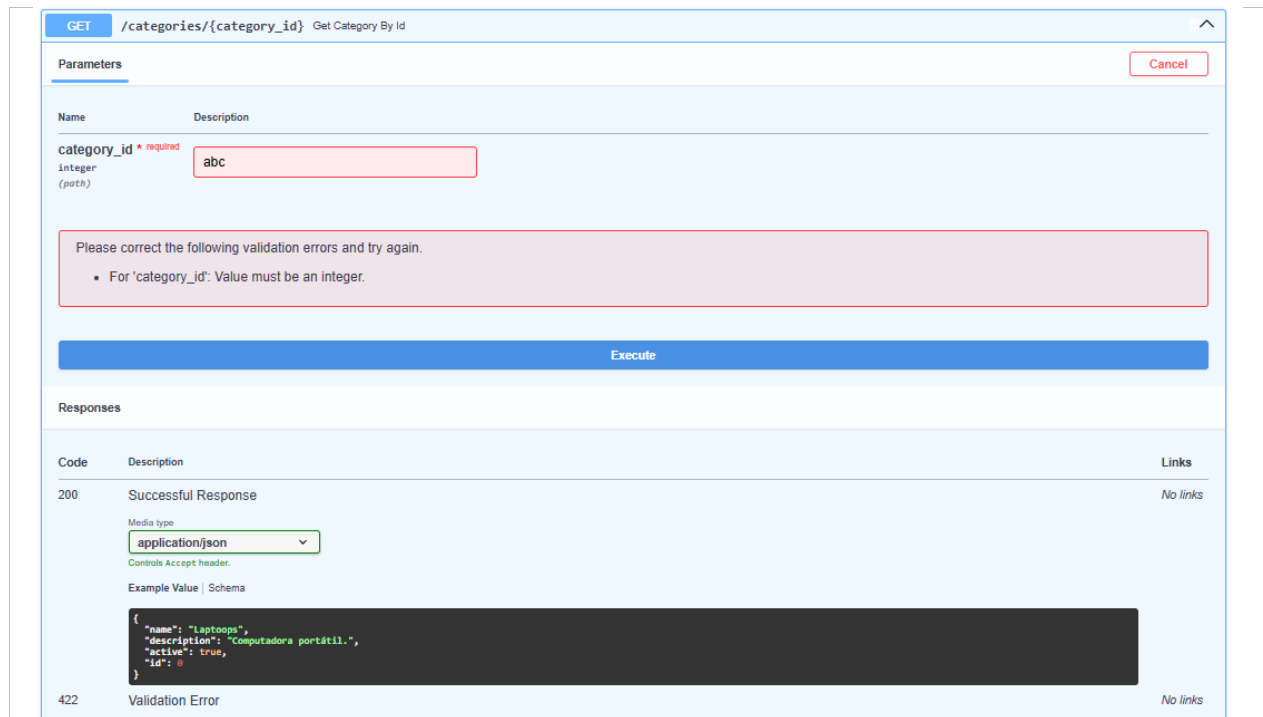


Figura — CA04 (1/2)

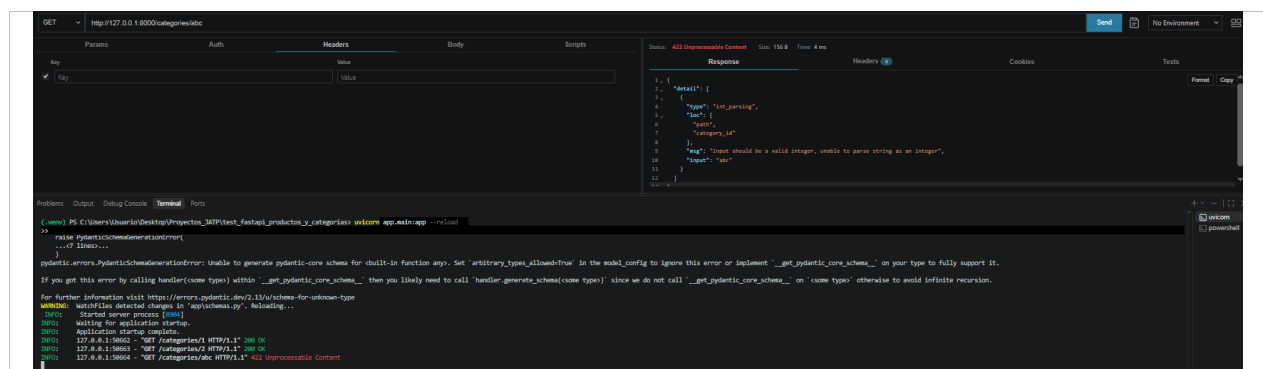


Figura — CA04 (2/2)

CA05 — Crear válida

Verifica la creación de una categoría con datos correctos.

Elementos que debe evidenciar la imagen:

- Se ve la petición POST `/categories` con el cuerpo de la solicitud.
- Se ve el código 201 y el registro creado en el log de la terminal.

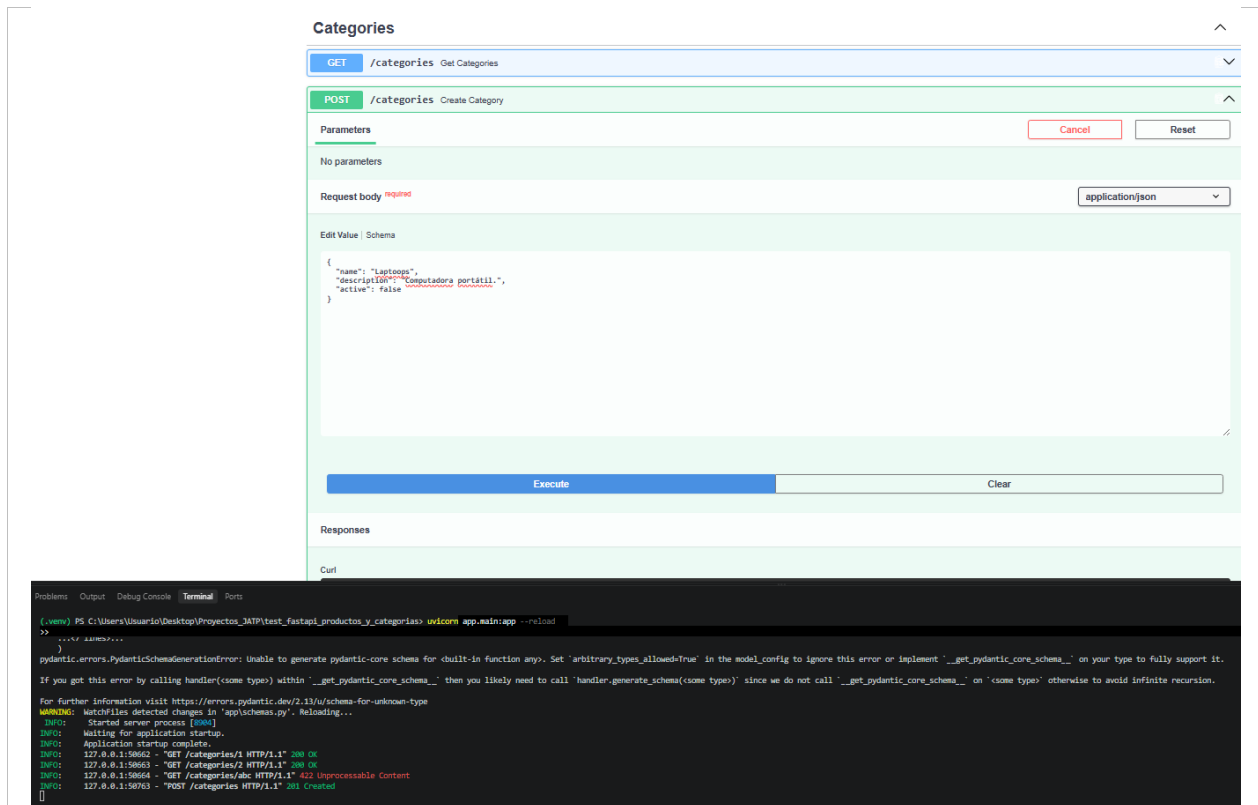


Figura — CA05

CA06 — Nombre demasiado corto

Verifica la validación de longitud mínima del campo name (menos de 3 caracteres).

Elementos que debe evidenciar la imagen:

- Se ve la petición POST /categories con un name de 1 a 2 caracteres.
- Se ve el código 422 con el mensaje "String should have at least 3 characters".

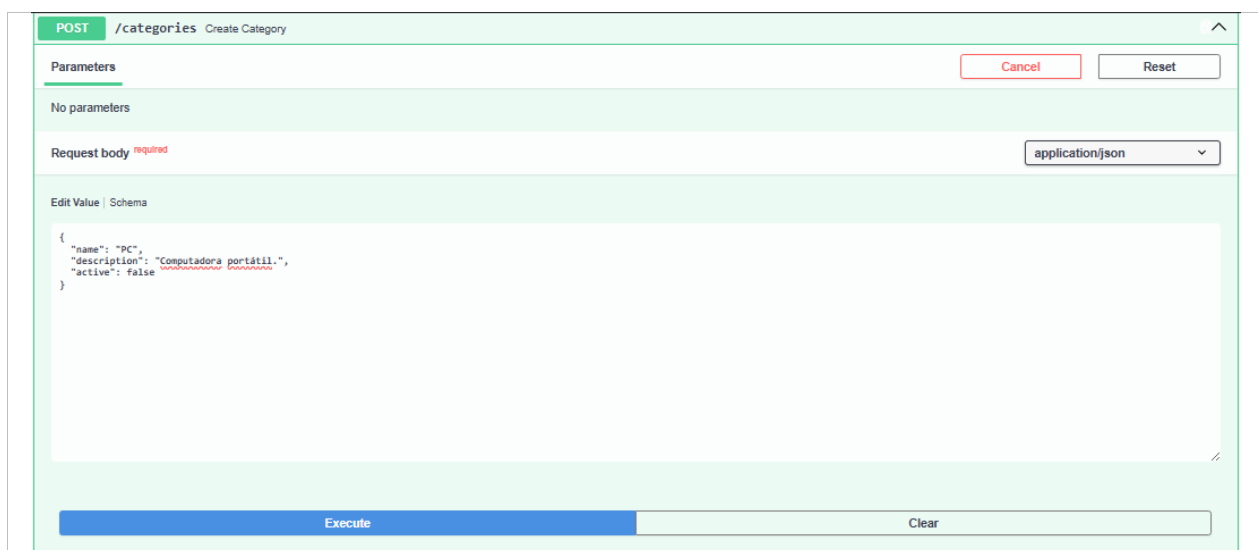


Figura — CA06 (1/2)

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8080/categories' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "pc",
    "description": "Computadora portátil.",
    "active": false
  }'
```

Request URL

```
http://127.0.0.1:8080/categories
```

Server response

Code	Details
422	Error: Unprocessable Content Response body <pre>{ "detail": [{ "type": "string_too_short", "loc": ["body", "name"], "msg": "String should have at least 3 characters", "input": "pc", "ctx": { "min_length": 3 } }] }</pre> Response headers <pre>content-length: 147 content-type: application/json date: Sun, 13 Sep 2026 23:20:45 GMT server: uvicorn</pre>

Responses

Code	Description	Links
201	Successful Response	No links

Figura — CA06 (2/2)

CA07 — Falta nombre

Verifica la validación cuando el campo name llega vacío.

Elementos que debe evidenciar la imagen:

- Se ve la petición POST /categories sin name (cadena vacía).
- Se ve el código 422 indicando el error de longitud mínima.

POST /categories Create Category

Parameters Cancel Reset

No parameters

Request body *required* application/json

Edit Value | Schema

```
{
  "name": "",
  "description": "Computadora portátil.",
  "active": false
}
```

Execute Clear

Responses

Figura — CA07 (1/2)

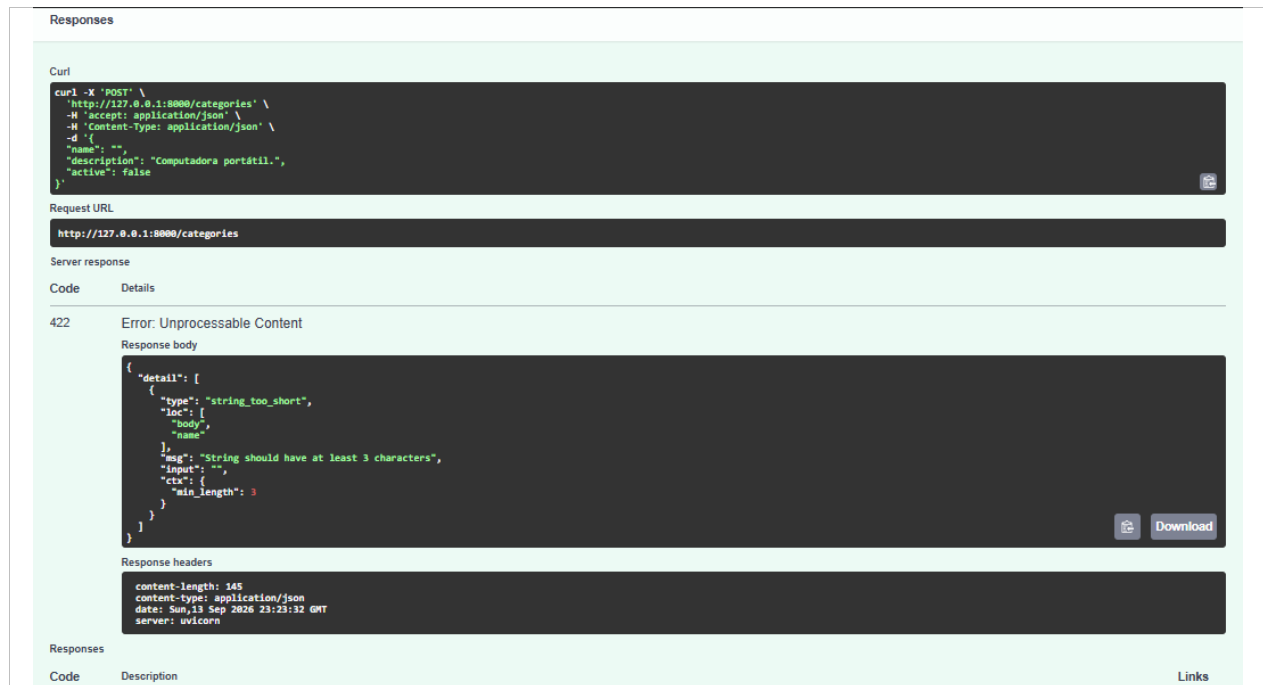


Figura — CA07 (2/2)

CA08 — Actualizar existente

Verifica la actualización parcial de una categoría existente.

Elementos que debe evidenciar la imagen:

- Se ve la petición PATCH /categories/1 con los nuevos datos.
- Se ve el código 200 con la categoría actualizada.

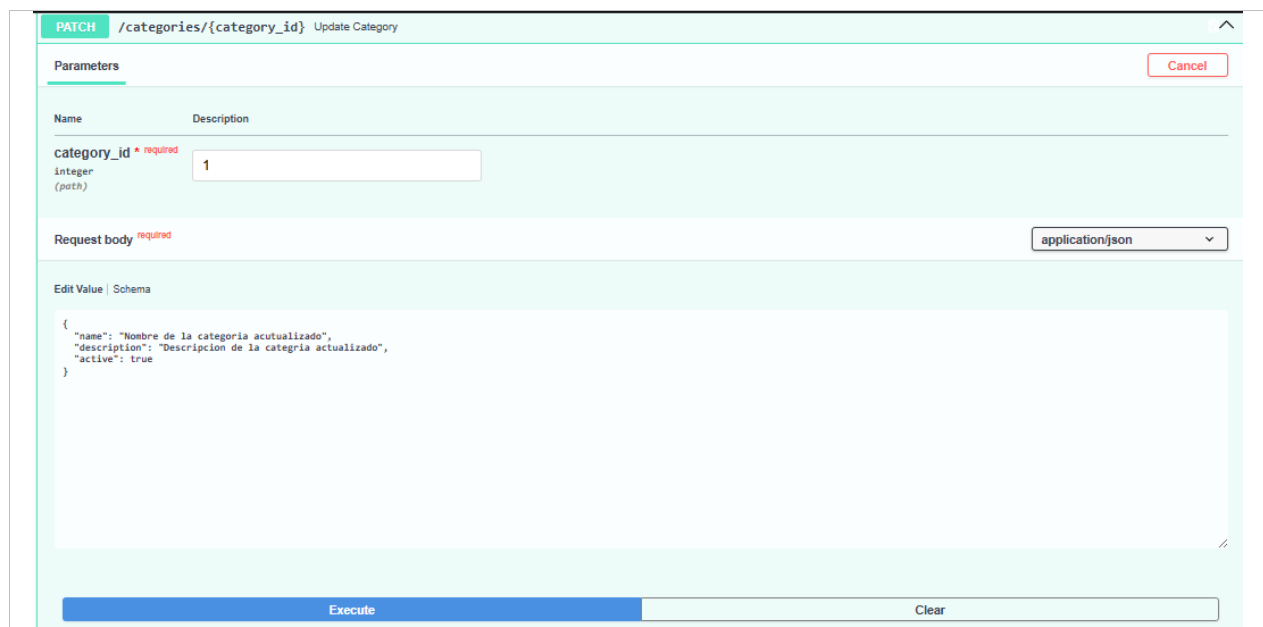


Figura — CA08 (1/2)

Responses

Curl

```
curl -X 'PATCH' \
  'http://127.0.0.1:8000/categories/1' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Nombre de la categoría actualizado",
    "description": "Descripción de la categoría actualizado",
    "active": true
  }'
```

Request URL

```
http://127.0.0.1:8000/categories/1
```

Server response

Code	Details
200	Response body <pre>{ "name": "Nombre de la categoría actualizado", "description": "Descripción de la categoría actualizado", "active": true, "id": 1 }</pre> Response headers <pre>content-length: 122 content-type: application/json date: Sun, 13 Sep 2026 23:26:36 GMT server: unicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Figura — CA08 (2/2)

CA09 — Actualizar inexistente

Verifica el manejo de error al actualizar una categoría que no existe (id = 0).

Elementos que debe evidenciar la imagen:

- Se ve la petición PATCH /categories/0.
- Se ve el código 404 con el detalle "Category not found".

PATCH /categories/{category_id} Update Category

Parameters

Name	Description
category_id * required	

integer (path)

0

Request body * required

application/json

Edit Value | Schema

```
{
  "name": "Nombre de la categoría actualizado",
  "description": "Descripción de la categoría actualizado",
  "active": true
}
```

Execute **Clear**

Responses

Figura — CA09 (1/2)

Responses

Curl

```
curl -X 'PATCH' \
  'http://127.0.0.1:8000/categories/0' \
  -H 'accept: application/json' \
  -H 'content-type: application/json' \
  -d '{
    "name": "Nombre de la categoría actualizado",
    "description": "Descripción de la categoría actualizado",
    "active": true
  }'
```

Request URL

http://127.0.0.1:8000/categories/0

Server response

Code	Details	Links
404	Error: Not Found	

Response body

```
{
  "detail": "Category not found"
}
```

Response headers

```
content-length: 31
content-type: application/json
date: Sun, 13 Sep 2026 23:28:55 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Figura — CA09 (2/2)

CA10 — Eliminar existente

Verifica la eliminación de una categoría existente.

Elementos que debe evidenciar la imagen:

- Se ve la petición DELETE /categories/1.
- Se ve el código 204 y el registro correspondiente en el log de la terminal.

DELETE /categories/{category_id} Delete Category

Parameters

Cancel

Name	Description
category_id * required	
integer (path)	1

Execute Clear

Responses

Curl

```
curl -X 'DELETE' \
  'http://127.0.0.1:8000/categories/1' \
  -H 'accept: */*'
```

Request URL

http://127.0.0.1:8000/categories/1

Server response

Code	Details	Links
204		

Response headers

```
date: Sun, 13 Sep 2026 23:30:25 GMT
server: uvicorn
```

Figura — CA10 (1/2)

```
(.venv) PS C:\Users\Usuario\Desktop\Proyectos_JATP\test_fastapi_productos_y_categorias> uvicorn app.main:app --reload
>>
WARNING: WatchFiles detected changes in 'app\schemas.py'. Reloading...
INFO: Started server process [8904]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:50662 - "GET /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50663 - "GET /categories/2 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50664 - "GET /categories/abc HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50763 - "POST /categories HTTP/1.1" 201 Created
INFO: 127.0.0.1:50770 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50775 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50799 - "PATCH /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50800 - "GET /categories HTTP/1.1" 200 OK
INFO: 127.0.0.1:50802 - "PATCH /categories/0 HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:50805 - "DELETE /categories/1 HTTP/1.1" 204 No Content
[]
```

Figura — CA10 (2/2)

CA11 — Eliminar inexistente

Verifica el manejo de error al eliminar una categoría que no existe (id = 0).

Elementos que debe evidenciar la imagen:

- Se ve la petición DELETE /categories/0.
- Se ve el código 404 con el detalle "Category not found".

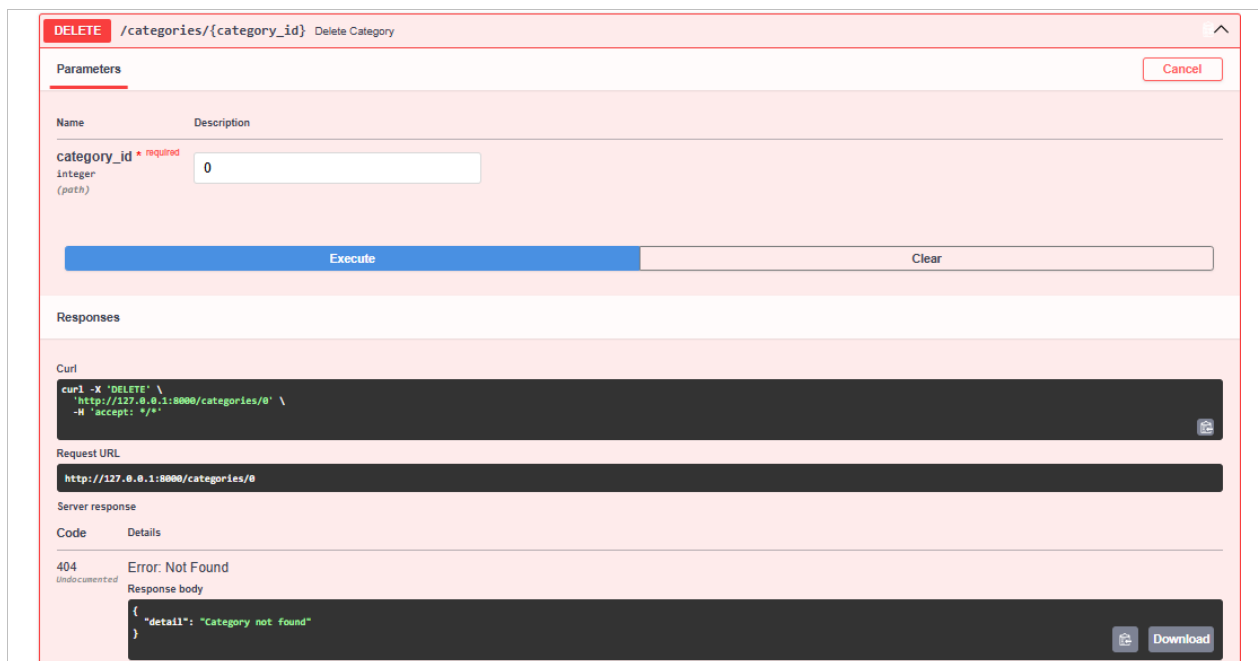


Figura — CA11 (1/2)

```

(.venv) PS C:\Users\Usuario\Desktop\Proyectos_JATP\test_fastapi_productos_y_categorias> uvicorn app.main:app --reload
>>
WARNING: Watchfiles detected changes in 'app\schemas.py'. Reloading...
INFO: Started server process [8904]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:50662 - "GET /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50663 - "GET /categories/2 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50664 - "GET /categories/abc HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50763 - "POST /categories HTTP/1.1" 201 Created
INFO: 127.0.0.1:50770 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50775 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50799 - "PATCH /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50800 - "GET /categories HTTP/1.1" 200 OK
INFO: 127.0.0.1:50802 - "PATCH /categories/0 HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:50805 - "DELETE /categories/1 HTTP/1.1" 204 No Content
INFO: 127.0.0.1:50811 - "DELETE /categories/0 HTTP/1.1" 404 Not Found

```

Figura — CA11 (2/2)

CA12 — Filtrar activas

Verifica el filtro ?active=true, que solo debe devolver categorías activas.

Elementos que debe evidenciar la imagen:

- Se ve la petición GET /categories?active=true.
- Se ve el código 200 con únicamente las categorías cuyo active es true.

GET /categories Get Categories

Parameters

Name	Description
active	boolean (boolean null) (query)
search	string (string null) (query)

active: true

search: search

Execute Clear

Figura — CA12 (1/2)

200

Response body

```
[
  {
    "name": "Smartphones",
    "description": "Teléfono celular inteligente.",
    "active": true,
    "id": 2
  },
  {
    "name": "Tablets",
    "description": "Tableta táctil.",
    "active": true,
    "id": 3
  },
  {
    "name": "Headphones",
    "description": "Audífonos para música.",
    "active": true,
    "id": 4
  },
  {
    "name": "Monitors",
    "description": "Pantalla para computadora.",
    "active": true,
    "id": 5
  },
  {
    "name": "Keyboards",
    "description": "Teclado para computadora.",
    "active": true,
    "id": 6
  }
]
```

Response headers

```
content-length: 417
content-type: application/json
date: Sun, 13 Sep 2026 23:33:48 GMT
server: uvicorn
```

Problems Output Debug Console Terminal Ports

```
(.venv) PS C:\Users\Usuario\Desktop\Proyectos_JATP\test_fastapi_productos_y_categorias> uvicorn app.main:app --reload
>>
WARNING: WatchFiles detected changes in 'app\schemas.py'. Reloading...
INFO: Started server process [8904]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: 127.0.0.1:50662 - "GET /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50663 - "GET /categories/2 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50664 - "GET /categories/abc HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50763 - "POST /categories HTTP/1.1" 201 Created
INFO: 127.0.0.1:50770 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50775 - "POST /categories HTTP/1.1" 422 Unprocessable Content
INFO: 127.0.0.1:50799 - "PATCH /categories/1 HTTP/1.1" 200 OK
INFO: 127.0.0.1:50800 - "GET /categories HTTP/1.1" 200 OK
INFO: 127.0.0.1:50802 - "PATCH /categories/0 HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:50805 - "DELETE /categories/1 HTTP/1.1" 204 No Content
INFO: 127.0.0.1:50811 - "DELETE /categories/0 HTTP/1.1" 404 Not Found
INFO: 127.0.0.1:50813 - "GET /categories?active=true HTTP/1.1" 200 OK
[]
```

Figura — CA12 (2/2)

Matriz de Casos de Prueba (CA01–CA12)

A continuación dejo la matriz de casos de prueba que me pide la guía de la actividad. La columna "Resultado" la lleno con PASS o FAIL según lo que me arrojó pytest -v al correr la suite completa.

ID	Escenario	Petición / condición	Esperado	Resultado
CA01	Listar categorías	GET /categories	200 y lista JSON	PASS
CA02	Consultar existente	GET /categories/1	200	PASS
CA03	Consultar inexistente	GET /categories/0	404	PASS
CA04	ID inválido	GET /categories/abc	422	PASS
CA05	Crear válida	POST con datos correctos	201	PASS
CA06	Nombre demasiado corto	POST con name de 1-2 caracteres	422	PASS
CA07	Falta nombre	POST sin name	422	PASS
CA08	Actualizar existente	PATCH /categories/1	200	PASS
CA09	Actualizar inexistente	PATCH /categories/0	404	PASS
CA10	Eliminar existente	DELETE /categories/1	204	PASS
CA11	Eliminar inexistente	DELETE /categories/0	404	PASS
CA12	Filtrar activas	GET /categories?active=true	200 y solo active=true	PASS

Lista de Verificación de Evidencias de Entrega

- Repositorio o carpeta del proyecto con el código fuente completo.
- Archivo requirements.txt.
- README.md con instrucciones para instalar, ejecutar la API y correr las pruebas.
- Captura de Swagger (/docs) mostrando todos los endpoints obligatorios.
- Captura o evidencia de la ejecución de pytest -v con la suite completa.
- Matriz de casos de prueba (CA01–CA12) diligenciada con el resultado obtenido: PASS o FAIL.

Observaciones Finales

Cuando corrí todas las pruebas con el comando `pytest -v`, me di cuenta que los doce casos de prueba (del CA01 al CA12) pasaron bien, todos salieron en PASSED y no me dio ningún error. Eso lo pude comprobar tanto en las capturas de Swagger como en la terminal.

Para las pruebas donde tenía que consultar algo que no existe (CA03, CA09 y CA11) use el id 0, porque en mi base de datos que tengo en memoria ese id tampoco existe, entonces me dio el resultado que se esperaba, que era el error 404 diciendo que la categoría no fue encontrada.

También pude hacer el reto opcional de la búsqueda por nombre, o sea el `?search=`, lo probé escribiendo la palabra "Mice" y la API sí me devolvió bien la categoría que tenía esa palabra, sin importar si la escribía con mayúscula o minúscula.

Aparte de eso, en mi proyecto también dejé hechos los endpoints de productos (`/products`) con sus pruebas, aunque esa parte no era obligatoria para esta actividad, pero quise dejarla porque ya la tenía hecha de antes.

En general puedo decir que la API me funcionó bien, cumplió con todos los códigos que pedía la guía (200, 201, 204, 404 y 422), las validaciones de Pydantic funcionaron como debían y las pruebas no dependieron del orden en que las ejecuté.